

# Assignment 3: Navigation with Expo Router and State Management

**Release Date:** October 12, 2025

**Due Date:** Sunday, October 26, 2025, 11:59 PM EST

**Weight:** 12.5% of final grade

## Objective

---

Extend the fitness tracker app from Assignment 2 by replacing React Navigation with Expo Router for file-based navigation and dynamic routes, and by implementing global state management using React's Context API with TypeScript. This assignment focuses on modern navigation patterns (dynamic routes for viewing and editing activities) and managing state globally to eliminate prop drilling. You will maintain all functionality from Assignment 2 (adding, updating, deleting, and displaying activities with input validation) while improving code organization and scalability. This builds skills in advanced React Native navigation and state management for mobile app development.

## Technologies #

---

- **React Native:** For building the mobile app UI and functionality.
- **TypeScript:** For type-safe development.
- **Expo CLI:** For project setup, development, and testing via Expo Go or emulators (use Node.js v22).
- **Expo Router:** For file-based navigation with dynamic routes.
- **React Context API:** For global state management of activities.

## Assignment Requirements

---

Build upon your Assignment 2 submission to implement a three-screen app using Expo Router and Context API. The screens remain the same (Home, Add/Edit, Details), but navigation will use file-based routing with dynamic routes for viewing and editing activities. The activities array will be managed globally via a Context provider instead of prop drilling. All styling, components, and functionality (add, update, delete, input validation) from Assignment 2 must remain intact, with the same in-memory `useState` storage.

### 1 React Native App Setup

- **Starter Code:** Begin with your completed Assignment 2 project as the foundation. Download the reference starter project by [clicking this link](#) to obtain `assignment-3.tar.gz`. This archive includes:

- Updated `package.json` and `package-lock.json` (adding Expo Router dependencies such as `expo-router`, `expo-status-bar`, and others)
- An updated `app.json` configured for Expo Router, including a deep linking `scheme` and support for statically typed routes
- A starter `app/` directory containing an `_layout.tsx` file

After downloading, extract the `.tar.gz` file, navigate to the project directory, and run:

```
npm install --legacy-peer-deps
```

to install all dependencies, and edit the necessary files to implement the required functionality.

- **Storage and Types:** Keep the in-memory activities array managed by `useState` (now in a Context provider). Update `types.ts` from Assignment 2 to support Expo Router and Context:
  1. Keep the `Activity` type.
  2. Add a `ActivityContextType` for the Context API, defining:
    - `activities : Activity[]`
    - `addActivity` : function taking an Activity without id
    - `updateActivity` : function taking an id and Activity without id
    - `deleteActivity` : function taking an id
  3. Remove `RootStackParamList`. Expo Router generates route and parameter types automatically by scanning the `app/` directory.
  4. Remove `NavigationProp`. It's not needed with Expo Router.
- **Context Setup:** Create a new `contexts/ActivityContext.tsx` file to manage the activities array globally:
  - Use `createContext` with `ActivityContextType`.
  - Implement a provider component ( `ActivityProvider` ) that uses `useState` to manage the `activities` array.
  - Provide functions ( `addActivity`, `updateActivity`, `deleteActivity` ) to modify the state, similar to Assignment 2's `setActivities` logic.
  - Export a custom hook ( `useActivities` ) to access the context in components.
  - Wrap the app in `ActivityProvider` in `app/_layout.tsx`.
- **Constants and Global Styles:** Reuse `constants/colors.ts` and `styles/globalStyles.ts` from Assignment 2 without changes. All screens must continue using `globalStyles.container` and `globalStyles.headerText` for consistency.

## 2 Custom Components

Reuse all custom components from Assignment 2 ( `Card.tsx`, `DetailsCard.tsx`, `ActivityListItem.tsx`, `PrimaryButton.tsx`, `ActionButton.tsx` ).

Only `ActivityListItem.tsx` requires changes:

- Replace the `setActivities` prop with the Context API (`useActivities`) to call `deleteActivity` and navigate to `details/[id]` or `add-edit` with `{ id }`.

All other components (`Card.tsx`, `DetailsCard.tsx`, `PrimaryButton.tsx`, `ActionButton.tsx`) remain unchanged.

### 3 Screen Implementations

Implement screens in the `app/` directory using Expo Router's file-based routing. Use the `useActivities` hook from the Context API for state access. Reuse components, styles, and validation logic from Assignment 2. Use `useRouter` and `useLocalSearchParams` from `expo-router` for navigation and route parameters.

- **Home Screen ( `app/index.tsx` ):**

- Same as Assignment 2's `HomeScreen.tsx` : Header ("Fitness Tracker") with `globalStyles.headerText`, `PrimaryButton` ("Add New Activity") to navigate to `add-edit`, `spacerView` (height 20), and `FlatList` rendering `ActivityListItem` for each activity with `keyExtractor` on `id`, `contentContainerStyle` for bottom padding (20), and `testID="activity-list"`.
- **Navigation:** When the "Add New Activity" button is pressed, navigate to the `add-edit` screen using the Expo Router API. In `ActivityListItem`, tapping an item should open its corresponding details screen.
- **State:** Access `activities` via the `useActivities` context instead of passing them as props.
- **Layout:** Wrapped in `globalStyles.container` with `testID="container"`.

- **Add/Edit Screen ( `app/add-edit.tsx` ):**

- Same as Assignment 2's `AddEditScreen.tsx` : Conditional header ("Add Activity" or "Edit Activity"), three `TextInput` fields (type, duration, calories), and `PrimaryButton` ("Add Activity" or "Update Activity") with input validation (same error messages).

#### Note

You can directly reuse the input validation logic you implemented in Assignment 1 and 2. While the autograder for Assignment 3 will not check input validation (since it is not the focus here), please keep it in your implementation for completeness.

- **Navigation:** Use `useLocalSearchParams` to retrieve the optional `id`.
  - If `id` exists, load the corresponding activity from context to pre-fill the form inputs.
  - On submit, call `addActivity` or `updateActivity` from the context, then navigate back to the Home screen using the Expo Router API.
- **State:** Use `useActivities` to access `activities`, `addActivity`, and `updateActivity`.
- **Styles:** Reuse Assignment 2's `input` style and `globalStyles.container`.

- **Details Screen ( `app/details/[id].tsx` ):**

- Same as Assignment 2's `DetailsScreen.tsx` : Header ("Activity Details") with `globalStyles.headerText` , `DetailsCard` for activity data, and a row of `ActionButtons` (Edit: `variant="primary"` , Delete: `variant="danger"` ).
- **Navigation:** Use `useLocalSearchParams` to get `id` and fetch activity from context. Edit navigates to `add-edit` with `{ id }` . Delete calls `deleteActivity` and navigates to `index` . Keep `testID="edit-button"` and `testID="delete-button"` .
- **State:** Use `useActivities` to access `activities` and `deleteActivity` .
- **Styles:** Reuse Assignment 2's button row styles.

## 4 Navigation with Expo Router

- **App Layout ( `app/_layout.tsx` ):** Wrap your app with the `AuthProvider` , and use the `Stack` component from `expo-router` to configure screen navigation. Apply the same header styling as in Assignment 2:
  - Header background: `colors.primary`
  - Header tint: `colors.white`
  - Title: bold and centered
  - Back title: `"Back"`
- **File Structure for Routes:**
  - `index.tsx` : Home screen
  - `add-edit.tsx` : Add/Edit screen
  - `details/[id].tsx` : Dynamic route for the Details screen
- **Screen Titles:** Set a title for each screen in the Stack configuration:
  - Home: `"Fitness Tracker"`
  - Add/Edit: Leave blank
  - Details: `"Activity Details"`
- **Dynamic Routes:** Use `[id]` in the file name for the Details screen to handle individual activity IDs. Access the `id` parameter using `useLocalSearchParams` .

## Testing Your Implementation

### Functionality Checklist

- ☐ Navigation uses Expo Router with file-based routing: `index` (Home), `add-edit` (Add/Edit), `details/[id]` (Details).
- ☐ State managed globally via `ActivityContext` with `useState` , providing `activities` , `addActivity` , `updateActivity` , `deleteActivity` .

- ☐ All Assignment 2 functionality (add, update, delete, validation) works with Context and dynamic routes.
- ☐ Styling unchanged from Assignment 2: `globalStyles`, `colors`, component styles.
- ☐ Navigation flows: Home → Add/Edit → Home, Home → Details → Home, Details → Edit → Home.
- ☐ App runs locally via Expo Go or emulator.

**Note**

The autograder will test navigation (dynamic routes), Context API usage, and component structure. Input validation from Assignment 2 must be preserved but will not be re-evaluated.

## Manual Testing

1. Start your app locally:

```
npx expo start
```

2. Test using:

- **Expo Go:** Scan the QR code with the Expo Go app.
- **Emulator:** Press `i` (iOS simulator) or `a` (Android emulator).
- **Inspector:** Press `m` to open the developer menu, then select "Show Element Inspector" to debug layout/styling.

## Sample Test Cases

Download `a3.sample.test.js` [here](#) and save it to your project directory. Run:

```
npm test
```

Review output to fix failing tests. Manually verify navigation and styling in Expo Go or an emulator. The sample tests cover 70% of the autograder's criteria.

## Submission Instructions

### 1 Generate the Archive

From the parent directory of your project, create a `.tar.gz` archive with:

```
cd /path/to/parent/directory
tar zcvf 1234567890-a3.tar.gz assignment-3
```

- Replace `assignment-3` with the name of your project directory.
- Replace `1234567890` with your student number.

- The archive must contain **exactly one top-level folder** (your project directory) with the required files.
- **Do not** include `node_modules/` , `.expo/` , or other generated files.

Your project directory (e.g., `assignment-3` ) should **exactly** match:

```
└─ assignment-3
  └─ app
    ├── _layout.tsx
    ├── index.tsx
    ├── add-edit.tsx
    └─ details
      └─ [id].tsx
  └─ app.json
  └─ assets
  └─ components
    ├── ActionButton.tsx
    ├── ActivityListItem.tsx
    ├── Card.tsx
    ├── DetailsCard.tsx
    └─ PrimaryButton.tsx
  └─ constants
    └─ colors.ts
  └─ contexts
    └─ ActivityContext.tsx
  └─ package-lock.json
  └─ package.json
  └─ styles
    └─ globalStyles.ts
  └─ tsconfig.json
  └─ types.ts
```

⚠ Any submission that fails to meet the specified format or structure and requires TA intervention for the autograder to work will receive a **20-point deduction**.

## 2 Verify Submission Structure

To avoid a 20-point deduction, use the provided Python script to check your `.tar.gz` file before submission.

1. [Download `check\_submission\_structure.py`](#) and save it in the parent directory of your project.
2. Ensure you have Python installed. Check by running `python --version` in your terminal/command prompt. If not installed, download it from [python.org](https://python.org) <sup>↗</sup>.
3. From the parent directory of your project, run:

```
python check_submission_structure.py 1234567890-a3.tar.gz
```

- Replace `1234567890` with your student number.

### 3 Submit to Quercus

Submit your `.tar.gz` file to [Quercus](#) <sup>↗</sup>.

#### Note

You are allowed to submit unlimited times. Only your latest submission before the deadline will be graded. Quercus automatically appends a suffix to the file name after the first submission — this will not affect grading.

## Grading Scheme (100 Points)

The autograder runs the app via `npx expo start`, testing navigation, Context API, and component usage. Partial marks are awarded for partial functionality. User input validation is not retested, but core functionality must work.

### App Setup and Context (15 pts)

- **Expo Router & Layout (9 pts):** `app/_layout.tsx` defines a `Stack` wrapped by `ActivityProvider` with correct `screenOptions` (header color, title style, etc.). *Partial marks:* structure (3), provider (3), `screenOptions` (3).
- **Types (6 pts):** `ActivityContextType` defines `activities`, `addActivity`, `updateActivity`, and `deleteActivity` (4). `Activity` type from Assignment 2 retained (2).

### Home Screen (22 pts)

- **Structure (10 pts):** `PrimaryButton` navigates to `/add-edit` (4), spacer `View` between sections (2), `FlatList` correctly lists `activities` with proper props (4).
- **Context (5 pts):** Uses `useActivities` to access `activities`.
- **Navigation (4 pts):** Tapping an `ActivityListItem` opens `details/[id]`.
- **Styling (3 pts):** Uses `globalStyles.container`, `globalStyles.headerText`, and reused Assignment 2 styles.

### Add/Edit Screen (22 pts)

- **Structure (9 pts):** Conditional header ("Add" vs. "Edit") (3); `PrimaryButton` adds or updates activity via context and returns to Home (6).
- **Context (5 pts):** Correct use of `useActivities` for adding/updating.
- **Navigation (5 pts):** Uses `useLocalSearchParams` for `id`, loads existing data when editing, navigates to Home on submit.

- **Styling (3 pts):** Reuses container, input, and button styles from Assignment 2.

## Details Screen (20 pts)

- **Structure (6 pts):** Header "Activity Details" (2); Edit/Delete buttons work correctly (4).
- **Context (5 pts):** Uses `useActivities` for loading/deleting activities.
- **Navigation (6 pts):** Reads `id` via `useLocalSearchParams`, navigates to `/add-edit` or `/`.
- **Styling (3 pts):** Reuses container, card, and button row styles from Assignment 2.

## Navigation Flows (21 pts)

- **Home → Add/Edit → Home (6 pts):** Add new activity and verify list updates.
- **Home → Details (4 pts):** Tap an activity to view details.
- **Details → Edit → Home (6 pts):** Edit activity, update reflected in Home list.
- **Details → Delete → Home (5 pts):** Delete activity, confirm list refresh.

## Resources

---

- **Course Materials:**
  - [Lecture 5: Navigation with Expo Router](#)
  - [Lecture 6: State Management in React Native](#)
- **External Resources:**
  - [Expo Router Documentation](#) ↗
  - [Learn React: Passing Data Deeply with Context](#) ↗

## Questions?

---

### 1. Discussion Board:

- Post questions on [course discussion board](#) ↗
- Search existing discussions first
- Use clear titles and provide relevant code snippets

### 2. Office Hours:

- Time: Mondays, 2:00 PM — 3:00 PM
- Location: Room 7206, Bahen Centre for Information Technology

### 3. Tips for Getting Help:

- Start early to allow time for questions
- Be specific about your problem

- Share what you've tried
- Include relevant error messages

Last updated on October 14, 2025